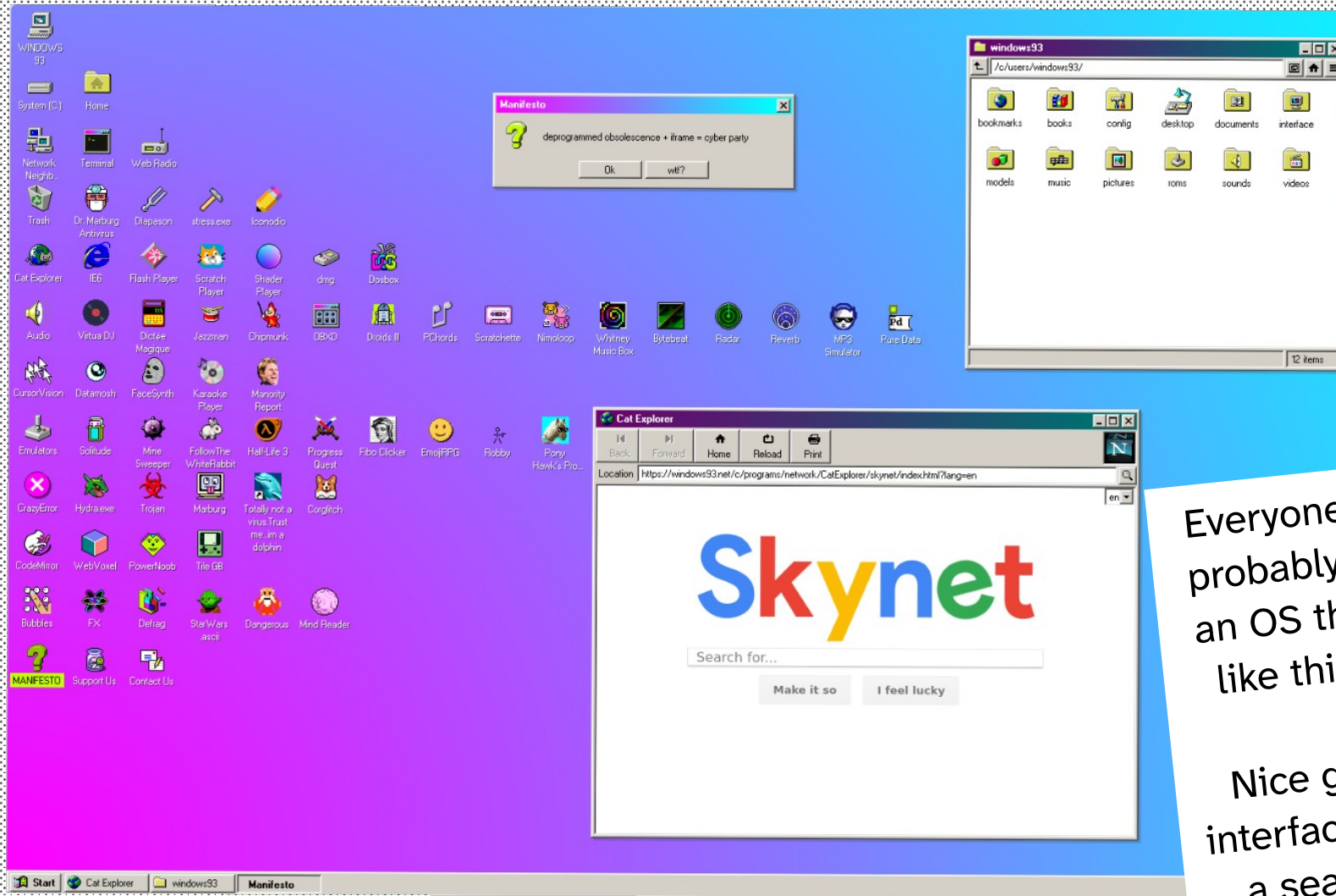


Software Tools

Lecture 01: Sysadmining

Gretchen Hallett



Everyone here is probably used to an OS that looks like this, right?

Nice graphical interface... icons... a search bar...

```
$ echo "The computers we're going to be  
using for this unit look rather  
different..."
```

```
$ echo "The computers we're going to be  
using for this unit look rather different..."  
The computers we're going to be using for  
this unit look rather different..  
$
```

DON'T PANIC

The shell is a **bit** scary at first... but you will get used to it

It is one of the oldest ways of interacting with a computer
and ONCE you get to grips with it is INCREDIBLY powerful;
if a bit eclectic

The trick to getting used to it is just to use it!

So lets have a look at the terminal!

```
$ ls -l --human-readable ~
```

So lets have a look at the terminal!

The prompt: the terminal is waiting for you to type a command

\$ ls -l --human-readable ~

Macs tend to use % instead of \$... (they use zsh instead of bash)

Some prompts get very fancy and give you extra information!

So lets have a look at the terminal!

```
$ ls -l --human-readable ~
```

The command is the first thing you type after the prompt. The `ls` command lists files.

So lets have a look at the terminal!

Arguments come after the command: this third one is a special symbol used to refer to your home folder on this computer

```
$ ls -l --human-readable ~
```

So lets have a look at the terminal!

Flags are special arguments that modify how the command works.

Short ones have one dash and a single letter

Long ones have two and a more readable name

```
$ ls -l --human-readable ~
```

(Usually! Other conventions exist)

(Java likes one dash for both!)

(Windows uses / instead of -!)

You can run it with <Return> !

So lets have a look at the terminal!

```
$ ls -l --human-readable ~  
-rwx----- 1 gretch bristol 4.6K Jun 24 14:54 nigel.jpg
```

The output is displayed below... its a picture of my cat!
Shall we open it?

So lets have a look at the terminal!

```
$ ls -l --human-readable ~  
-rwx----- 1 gretch bristol 4.6M Jun 24 14:54 nigel.jpg  
$ cat nigel.jpg  
/\-/\ The terminal isn't good at pictures, sadly...  
( o.o ) ...just text  
>^<
```

Some modern terminals (and even a few really old ones) can do images...
Search for the *kitty-graphics-protocol* or *sixel*s to find out more!

Wait a minute... How am I possibly meant to know that the command to see what files I have happens to be `ls`?

Isn't it `dir` on Windows? This doesn't seem intuitive...
Infact, it seems borderline hostile to users...

We are going to come back to the shell properly next week...

You are going to have to learn it
IT IS EXAMINABLE
(and will be on there)

It is going to take a few weeks of use...
Just immerse yourself and it will come...



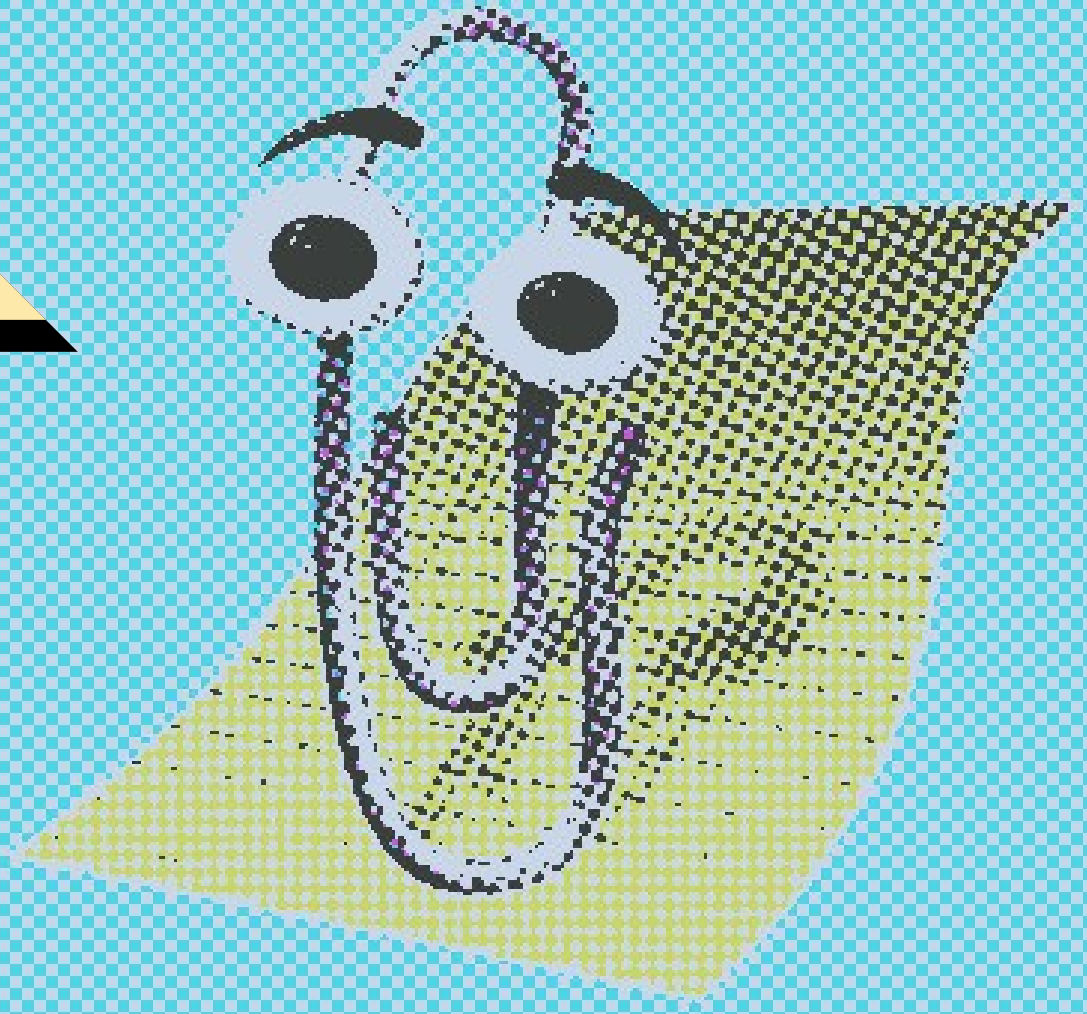
Also there is a manual to tell you how everything in your system works (that is specific to **your** system and not general help like Google and StackOverflow give you...)

Run `man man` to see the man tools manual... press `q` to quit, `/` to search, `j/k` to scroll

Run `man -k` or `apropos` to search the manual for things

And see `man intro` for beginner stuff!

The man pages are great...
If a little old fashioned



While there are standards that describe how a system should behave...

Very few systems follow them precisely... most vary a little... some vary a lot.

We will try and stick to POSIX with the lab machines as our (examinable) standard...

If you try and complete this unit with something that isn't a lab machine you are going to be sad

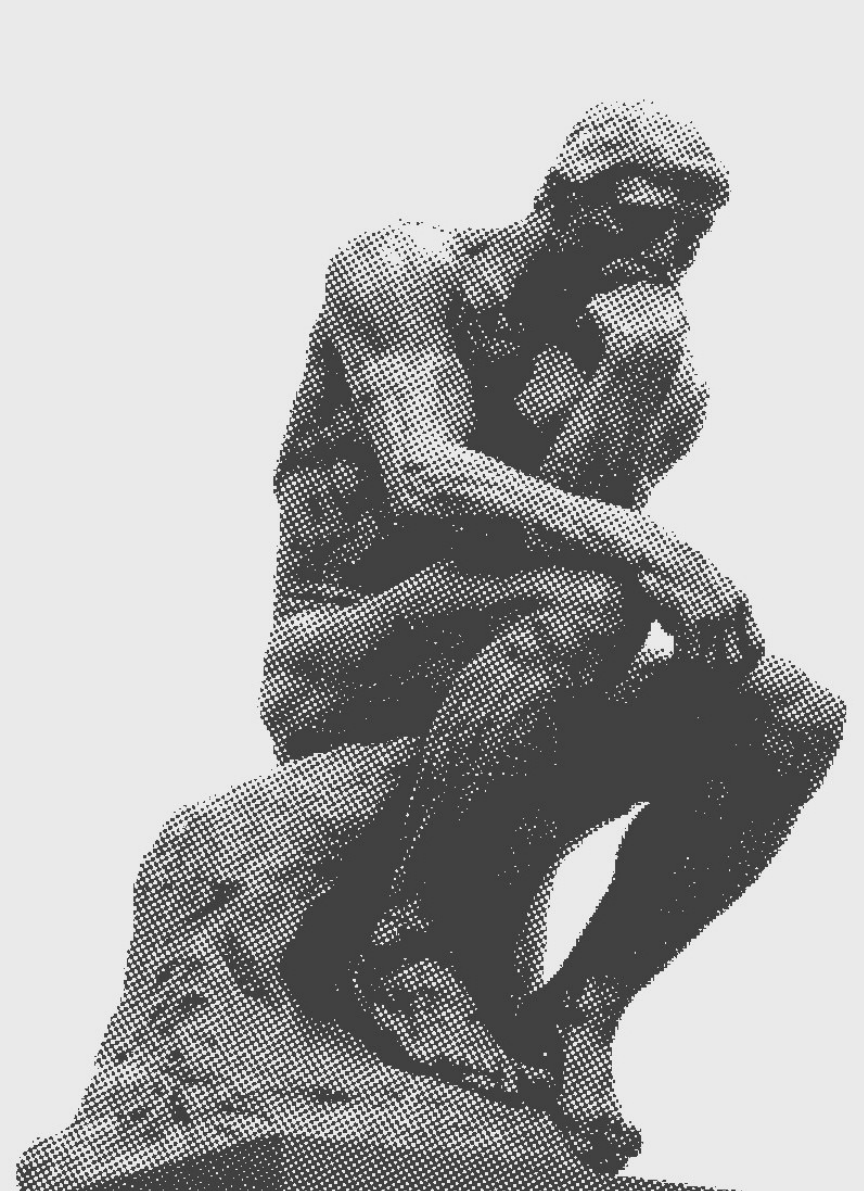
If you're running Linux
- It'll probably be fine...

If you're running a Mac most stuff will work EXCEPT:
- Virtualization (virtualbox/vagrant will not work)
- The debugging lab

If you have Windows
- WSL will *sometimes* work (but break in weird ways)
- Powershell / CMD.EXE will never work



The TAs are explicitly told NOT to help you with non-lab systems... just use the lab machine



So how am I going to complete this unit on my shiny new gaming laptop if I don't want to touch the filthy, dirty lab machines that are gross...

A) I was an undergrad here (20 years ago)
The machines are pretty much the same.

B) and so we come to our first *software tool*...



The SSH protocol lets you login and run commands on remote computers. It runs on **port 22**

OpenSSH is the most common implementation of the protocol; developed alongside the OpenBSD operating system

If you want to make (secure) connections between two or more computers it is **staggeringly** useful...

```
$ ssh evelyn@imac
```

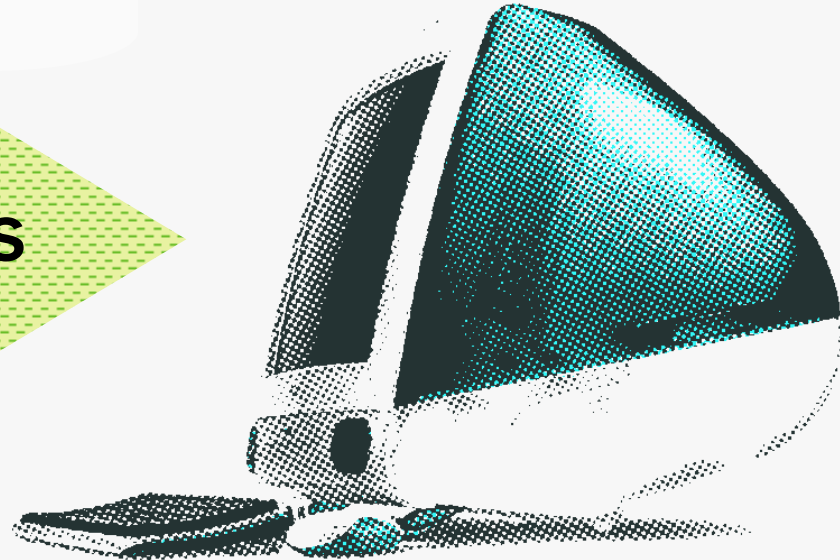
```
$ ssh -X evelyn@imac firefox
```

Evelyn

See `man sftp` or `man scp` if you
want to copy files remotely instead
(or `kermi t`)

imac

MAGIC OF NETWORKS



Actually the **first** time you connect it'll say...

```
The authenticity of host 'sdf.org (205.166.94.16)'  
can't be established.  
ED25519 key fingerprint is:  
SHA256:Zjwb07AU8rHJExYrmZS2LqGZ7WfdoELfMrF54W92PYA  
This key is not known by any other names.  
Are you sure you want to continue connecting  
(yes/no/[fingerprint])?
```

Get a friend with **physical access to the machine** to run...

```
$ ssh-keyscan -q -t ed25519 localhost
```

...to check

Why might you want to check the key? Seems pretty pointless?

Normally yes, but you can do some neat evil with OpenSSH

Smith tricks Evelyn into connecting to them instead of the iMac
Resends everything Evelyn sends to the iMac and sends responses
...but maybe records everything that happens
...and maybe tweaks somethings!

Don't look up
arp spoofing
as a means
for doing
this!



And then you'll get a shell...
So what are you meant to do with this system?

```
$ tree -L 1 /  
/  
├── bin -> usr/bin  
├── boot  
├── dev  
├── etc  
├── home  
├── lib -> usr/lib  
├── lost+found  
├── media  
├── mnt  
├── opt  
├── proc  
├── root  
├── run  
├── sbin -> usr/sbin  
├── srv  
├── sys  
├── tmp  
├── usr  
└── var
```

Binary programs go in bin/
Bootloader stuff goes in boot/
Device files (raw disks) are in /dev/
Configuration goes in etc/
Programs for the root user go in sbin/
Temporary files go in /tmp
Files being served go in srv/ or var/

run/ is for runtime things on Linux

/usr contains the same again but its
stuff that your OS thinks that you need
rather than the base OS (Linux doesn't
use this so /bin/ and /usr/bin are the
same but not /etc for no good reason)

/usr/local contains the same again but
its stuff you've locally installed on this
specific machine.

/opt might contain the same.

*Yes we know this makes no sense...
It's old and evolved over time*

On my OpenBSD machine it's different again

```
$ /usr/local/bin/tree -L 1 -f /  
/  
|-- /altroot      $ mount  
|-- /bin          /dev/sd1a on / type ffs (local)  
|-- /bsd          /dev/sd1k on /home type ffs (local, nodev, nosuid)  
|-- /bsd.booted   /dev/sd1d on /tmp type ffs (local, nodev, nosuid)  
|-- /bsd.rd       /dev/sd1f on /usr type ffs (local, nodev)  
|-- /bsd.sp       /dev/sd1g on /usr/X11R6 type ffs (local, nodev)  
|-- /dev          /dev/sd1h on /usr/local type ffs (local, nodev, wxallowed)  
|-- /etc          /dev/sd1j on /usr/obj type ffs (local, nodev, nosuid)  
|-- /home         /dev/sd1i on /usr/src type ffs (local, nodev, nosuid)  
|-- /mnt          /dev/sd1e on /var type ffs (local, nodev, nosuid)  
|-- /root  
|-- /sbin  
|-- /sys -> usr/src/sys  
|-- /tmp  
|-- /usr  
`-- /var
```

A Mac will be different again
As will WSL
Just know the *rough* conventions

Package Managers and APT

On Linux we have multiple ways to install new software. We can compile and install it manually (`./configure; make; make install`)

We can download and install prebuilt software manually (don't do this, but if you must in `/opt`)

Or we can ask the OS to get software and install it for us all automatically. The tool we need for this on the VMs we give you in the labs is called `apt`

It is the default package manager for Debian Linux (and anything derived from it like Ubuntu or Windows Subsystem for Linux)



The other package manager you'll see is `dnf` for RPM based distributions of Linux. Many (*many many*) others exist
Macs have a thing called `brew.sh` that you should **absolutely** look into

```
$ apt search 'common lisp compiler'
elpa-gcl27/stable,stable 2.7.1-4 all
  GNU Common Lisp compiler

gcl/stable 2.6.14-19 arm64
  GNU Common Lisp compiler

gcl-doc/stable,stable 2.6.14-19 all
  Documentation for GNU Common Lisp

gcl27/stable 2.7.1-4 arm64
  GNU Common Lisp compiler

gcl27-doc/stable,stable 2.7.1-4 all
  Documentation for GNU Common Lisp

gcl27-tk/stable 2.7.1-4 arm64
  GNU Common Lisp compiler

gcl27-tk-doc/stable,stable 2.7.1-4 all
  Documentation for GNU Common Lisp

sbcl/stable 2:2.5.2-1 arm64
  Common Lisp compiler and development system
```

```
$ apt install sbcl
Error: Could not open lock file
/var/lib/dpkg/lock-frontent - open (13:
Permission denied)
Error: Unable to acquire the dpkg frontend
lock (/var/lib/dpkg/lock-frontent), are you
root?
```

```
$ sudo !!
sudo apt install sbcl
Installing:
  sbcl
```

!! is bash for the
whole of the previous
command typed...
more shortcuts exist!

```
Suggested packages:
  sbcl-doc  sbcl-source  slime
```

```
Summary:
  Upgrading: 0, Installing: 1, Removing: 0,
  Not Upgrading: 3
  Download size: 8,885 kB
  Space needed: 41.4 MB / 22.6 GB available
```

```
Get:1 http://deb.debian.org/debian
trixie/main arm64 sbcl arm64 2:2.5.2-1 [8,885
kB]
```

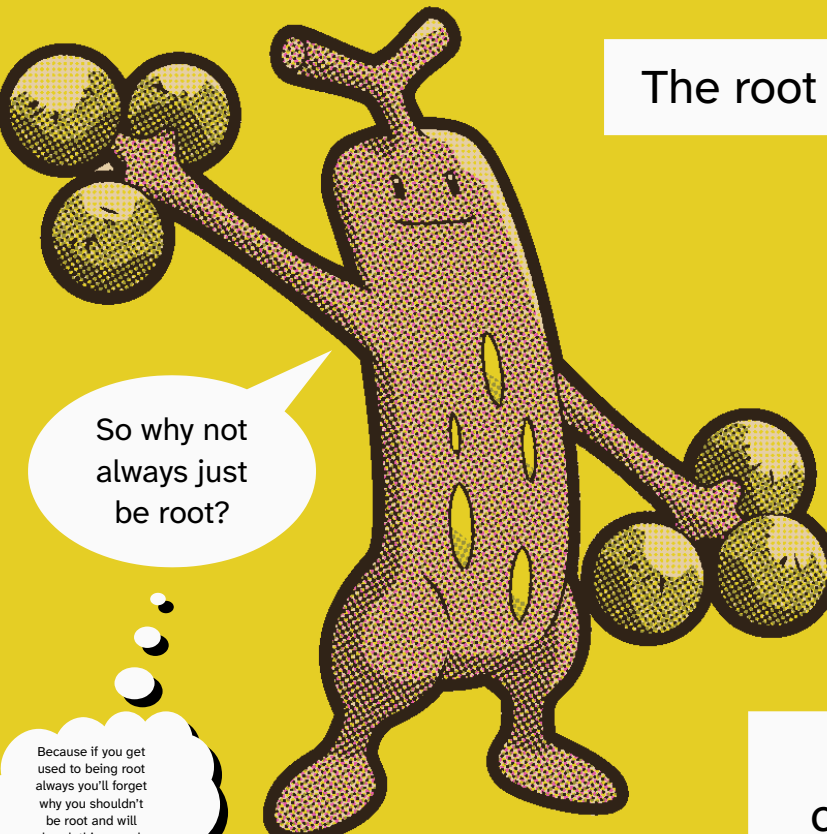
Linux (and most UNIXes) have a concept of users, where each user only has control over the minimum things they need to work

The root user (uid=1, Charlie Root) is all powerful

If you want to do something that could mess up the system for other users (like installing/removing/changing) software...

Only the root user is allowed to do it.

You can **temporarily** become root with the sudo command (or doas... or pk-exec... or su -c... or ...)



So why not always just be root?

Because if you get used to being root always you'll forget why you shouldn't be root and will break things and swear a lot.

Users, groups, and the UNIX DAC

UNIX systems *traditionally* control access to everything through files.

Almost everything is treated as a file which you can read, write or execute (RWX).

Every file is owned by exactly one user and one group (see them with `ls -l`).

All users belong at least one group.

Permission to read, write or execute any file is granted based on the user, the group, and for everyone else

And they traditionally like to express these permissions as octal number for no good reason.

Users, groups, and the UNIX DAC

UNIX systems *traditionally* control access to everything through files.

(...but other mechanisms exist nowadays too)

Almost everything is treated as a file which you can read, write or execute (RWX).

(except directories, daemons, and some kernel memory structures)

Every file is owned by exactly one user and one group (see them with `ls -l`).

```
-rwxr-x--- root wheel my-cool-script.sh
```

All users belong at least one group.

uid=1000(goblin) gid=1000(goblin) groups=1000(goblin), 0(wheel), 9(wsrc), 1002(_config),

Permission to read, write or execute any file is granted based on the user, the group, and for everyone else

(So I'd be able to read and execute the script but not write)

And they traditionally like to express these permissions as octal number for no good reason.

$RWX = 111_2 = 7_8$, $R-x = 101_2 = 5_8$, $--- = 000_2 = 0_8$

Users, groups, and the UNIX DAC

Use `chmod` to change permissions (see man page for modern non-octal syntax)

Use `chown` to change owning user and groups

Directories are a bit weird

- If you want to be able to `cd` into a directory you need the execute bit set
- If you want to be able to read the contents of a directory you need the read bit set

There are also special bits for files:

- If the `setuid` bit is set on an executable it runs as the owning user not the executing user (*this is how `sudo` works*)
- If a directory has the `setgid` bit set new files are owned by the group of the directory not the group of the creating process
- If the sticky bit is set then only root or the directory's owner can delete (unlink) files

Users, groups, and the UNIX DAC

The directory of what users and groups exist and who belongs to what is kept in /etc/passwd and /etc/group; user passwords are *sometimes* kept in /etc/shadow:

```
$ grep goblin /etc/passwd
goblin:*:1000:1000:Gretchen
Hallett,,,:/home/goblin:/usr/local/bin/bash
```

```
$ grep wheel /etc/group
wheel:*:0:root,goblin
```

```
$ sudo grep goblin /etc/shadow
goblin:$y$jB5$dd4SiZK0lEYBmM4PAdo9q.
$mLrcNj9S.IBwLFoYxkL0Ik8POhU2UM1xXGUNnzK8Hj4:20426:0:99999:7:::
```

See the man pages (particularly section 5).

Right so we can login...

We can install software...

What are we going to do with it?

Most people don't use a computer for the sheer joy of computing...

Well we probably want to run some things to let us do more interesting things!

gophernicus

Like a web/gopher/gemini/finger server to host our art!

An IRC server so we can chat with our friends!

unrealIRCd

An IRC bouncer so we can keep logged into all our IRC servers!

soju

snac

A mastodon server so we can have private social media with no ads for our friends!

A streaming server so we can share all our legitimately acquired media!

Jellyfin

A Git server to share our code with others!

A podcast aggregator so we can listen to radio!

audiobookshelf

A Minecraft server so you don't have to long for the mines!

I, personally, do not understand the appeal of Minecraft. I was in uni when the first betas appeared and couldn't get into it... that you (and my kids) are playing a 15 year old game still boggles my mind

We **can** do all these things by just running the programs...

But its easier *and* **safer** to have them managed by the OS

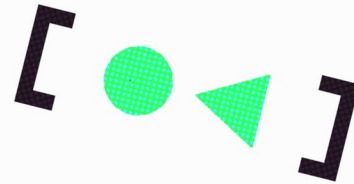
Now lets ignore the fact that I **still** play Pokemon and Super Mario World ROM hacks...

There are **several** service managers for Linux...

The “*normal*” one is Lennart Poettering’s systemd

Systemd is *polarizing*...

- It replaced a bunch of shellscripts with Microsoft-style system management
- It breaks UNIX/POSIX tradition (sometimes for a good reason, sometimes not)
- It tries to standardize Linux (which goes against what makes it fun)
- The project has *massive* scope-creep
- You can automate a lot with it once you bother to learn it
- It requires you to use Linux and not other OSs



NB. I think my view is it's fine but its taken a long time to get here, and I'm glad I don't normally run Linux and have to use it day to day...

Systemd wants to manage
everything... its not 453 real things

```
$ systemctl status
```

```
● gretchs-debian-server
```

```
State: running
```

```
Units: 453 loaded (incl. loaded aliases)
```

```
Jobs: 0 queued
```

```
Failed: 0 units
```

```
Since: Thu 1970-01-01 01:00:04 BST; 56 years 7 months ago
```

```
systemd: 257.13-1~deb13u1
```

```
Tainted: unmerged-bin
```

```
CGroup: /
```

```
└─init.scope
```

```
└─┬1 /sbin/init
```

```
└─system.slice
```

```
└─┬ModemManager.service
```

```
└─┬127933 /usr/sbin/ModemManager
```

```
└─┬NetworkManager.service
```

```
└─┬127921 /usr/sbin/NetworkManager --no-daemon
```

Ignore this
(systemd
scope creep)

WTF?

```
lines 1-16
```

```
$ journalctl -n 3                                # show last three system logs
Aug 12 14:55:54 gretchs-server sudo[128327]: pam_unix(sudo:session):
session closed for user root
Aug 12 14:56:13 gretchs-server tailscaled[76077]: control: NetInfo:
NetInfo{varies=false ipv6=false ipv6os=true udp=true icmpv4=false derp=#8
port>
Aug 12 14:56:13 gretchs-server tailscaled[76077]: magicsock: endpoints
changed: 80.0.38.136:6007 (stun), 80.0.38.136:41641 (stun),
80.0.38.136:600>

$ systemctl start sshd                          # turn sshd on
$ systemctl enable sshd                         # run it at boot
$ systemctl enable --now sshd                  # turn it on and run it at boot
$ journalctl -u sshd --since '10min ago'        # see logs from ten mins ago
-- No entries --
$ journalctl -u sshd --boot                     # see logs since boot
-- No entries --
# grep -R sshd /var/log/                        # still nothing?
```

```
$ systemctl status sshd
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-08-12 13:25:47 BST; 1h 26min ago
 Invocation: 8f09b4464b91443eba94e661e6505275
    Docs: man:sshd(8)
          man:sshd_config(5)
 Main PID: 126792 (sshd)
   Tasks: 1 (limit: 8796)
    CPU: 250ms
   CGroup: /system.slice/ssh.service
           └─126792 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Aug 12 13:25:47 gretchs-server systemd[1]: Starting ssh.service - OpenBSD Secure
Shell server...
Aug 12 13:25:47 gretchs-server sshd[126792]: Server listening on 0.0.0.0 port 22.
Aug 12 13:25:47 gretchs-server sshd[126792]: Server listening on :: port 22.
Aug 12 13:25:47 gretchs-server systemd[1]: Started ssh.service - OpenBSD Secure
Shell server.
Aug 12 14:30:38 gretchs-server sshd-session[128089]: Accepted publickey for goblin
from 100.86.61.81 port 38105 ssh2: ED25519 SHA256:IfKZuXbSEleOp>
```

```
$ cat /etc/systemd/system/sshd.service
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8) man:sshd_config(5)
After=network.target nss-user-lookup.target auditd.service
ConditionPathExists=!/etc/ssh/sshd_not_to_be_run
```

```
[Service]
EnvironmentFile=-/etc/default/ssh
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D $SSHD_OPTS
ExecReload=/usr/sbin/sshd -t
ExecReload=/bin/kill -HUP $MAINPID
KillMode=process
Restart=on-failure
RestartPreventExitStatus=255
Type=notify
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755
```

```
[Install]
WantedBy=multi-user.target
Alias=sshd.service
```

```
$ sudo dmesg -H # get the kernel's log (if you're still stuck)
[Jul27 13:52] Booting Linux on physical CPU 0x000000000000 [0x410fd083]
[ +0.000000] Linux version 6.18.34+rpt-rpi-v8 (serge@raspberrypi.com) (aarch64-linux-gnu-gcc-14
(Debian 14.2.0-19) 14.2.0, GNU ld (GNU Bin>
[ +0.000000] KASLR enabled
[ +0.000000] random: crng init done
[ +0.000000] Machine model: Raspberry Pi 4 Model B Rev 1.4
[ +0.000000] efi: UEFI not found.
[ +0.000000] Reserved memory: created CMA memory pool at 0x0000000000e400000, size 512 MiB
[ +0.000000] OF: reserved mem: initialized node linux,cma, compatible id shared-dma-pool
[ +0.000000] OF: reserved mem: 0x0000000000e400000..0x000000002e3ffffff (524288 KiB) map reusable
linux,cma
[ +0.000000] OF: reserved mem: 0x0000000003ef74d20..0x0000000003ef74e01 (0 KiB) nomap non-reusable
nvram@0
[ +0.000000] OF: reserved mem: 0x0000000003ef74ae0..0x0000000003ef74cdf (0 KiB) nomap non-reusable
nvram@1
[ +0.000000] NUMA: Faking a node at [mem 0x0000000000000000-0x000000001fffffffff]
[ +0.000000] Faking node 0 at [mem 0x0000000000000000-0x00000000103ffffff] (4160MB)
[ +0.000000] Faking node 1 at [mem 0x00000000104000000-0x000000001fffffffff] (4032MB)
[ +0.000000] NODE_DATA(0) allocated [mem 0x103ffd280-0x103ffffff]
[ +0.000000] NODE_DATA(1) allocated [mem 0x1feff6280-0x1feff8fff]
[ +0.000000] Zone ranges:
[ +0.000000]   DMA      [mem 0x0000000000000000-0x0000000003ffffff]
[ +0.000000]   DMA32    [mem 0x00000000040000000-0x000000000ffffffff]
[ +0.000000]   Normal   [mem 0x00000000100000000-0x000000001ffffffff]
```

We have covered:

- basic shell syntax
- how to log on to the lab machines remotely
 - hinted at *man-in-the-middle* attack
- discussed common filesystem hierarchy
 - the package manager
 - the sudo command
 - the UNIX DAC
 - users and groups
 - starting services
- getting access to the logs



Bamboozle

Bamber Boozler poses another 3 questions. Answer using
fasttext at the bottom. One wrong and you must try again.
Can you answer all 3 in one attempt?

Good day. Let's make it
not a blue, but a
musical, Monday with a
dozen posers covering
all styles and tastes.
Ready?



Press RED to begin

-----1/1--
Start

Guide

Turner The Worm

Digitiser

P45A

Digitext

45A

Aug 12

15:18:28

What UNIX command do you need to log in to a remote machine, and what port does it run on?



Red:ssh, 80

Green:ssh, 22

Yellow:systemctl sshd start, 22

Cyan:sudo ssh, 79

????

????

????

????

1/1

P45A

Digitext

45A

Aug 12

15:21:47

A file is owned by brian (group users), and has permissions 0754. Can nigel (group users) read it and can he edit it?



Red: no, no

Green: no, yes

Yellow: yes, no

Cyan: yes, yes

????

????

????

????

1/1

Nigel would like to run the httpd web server on their computer. How should they start it with systemd?



Red:systemctl enable httpd

Green:httpd

Yellow:systemctl start httpd

Cyan:rcctl start httpd

????

????

????

????